PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering Lab (PES_SE_B_2026B)

Lab 6: Code and Branch Coverage Analysis

Student Name:	Deeksha G
SRN:	PES1UG24AM078
Assigned Program:	student_grades.py (Repo 16)
GitHub Repository:	https://github.com/DEEKSHAGURURAJA/L6_L7_16_student_grades

1. Objective

To measure, analyze, and achieve target test coverage on the assigned Python module (student_grades.py) using Python's coverage and pytest tools. The primary goals are to understand the practical difference between statement (line) coverage and branch (decision) coverage, achieve at least 90% in both, and commit the solutions according to the lab rubric.

2. Program Under Test (student_grades.py)

The application contains three core functions with multiple conditional branches: • **letter_grade(marks):** Converts marks (0 to 100) into letter grades (S, A, B, C, D, E, F). Raises ValueError for out-of-range marks (< 0 or > 100).
• **final_marks(internal, external, has_project):** Sums internal (max 40) and external (max 60) marks. Adds a 5-mark project bonus if applicable, capped at 100.
• **exam_eligibility(attendance, fees_paid, has_medical):** Evaluates student eligibility ('eligible', 'detained', or 'blocked') based on fee status, attendance percentage, and medical condonation.

3. Summary of Deliverables

• **Code Coverage Suite:** CodeCoverageTest_student_grades.py — achieves 100% statement coverage.
• **Branch Coverage Suite:** BranchCoverageTest_student_grades.py — achieves 100% branch coverage (26/26 branches).
• **Version Control:** Two distinct commits created and pushed upstream to GitHub.

4. Part 1: Code Coverage (Statement Coverage)

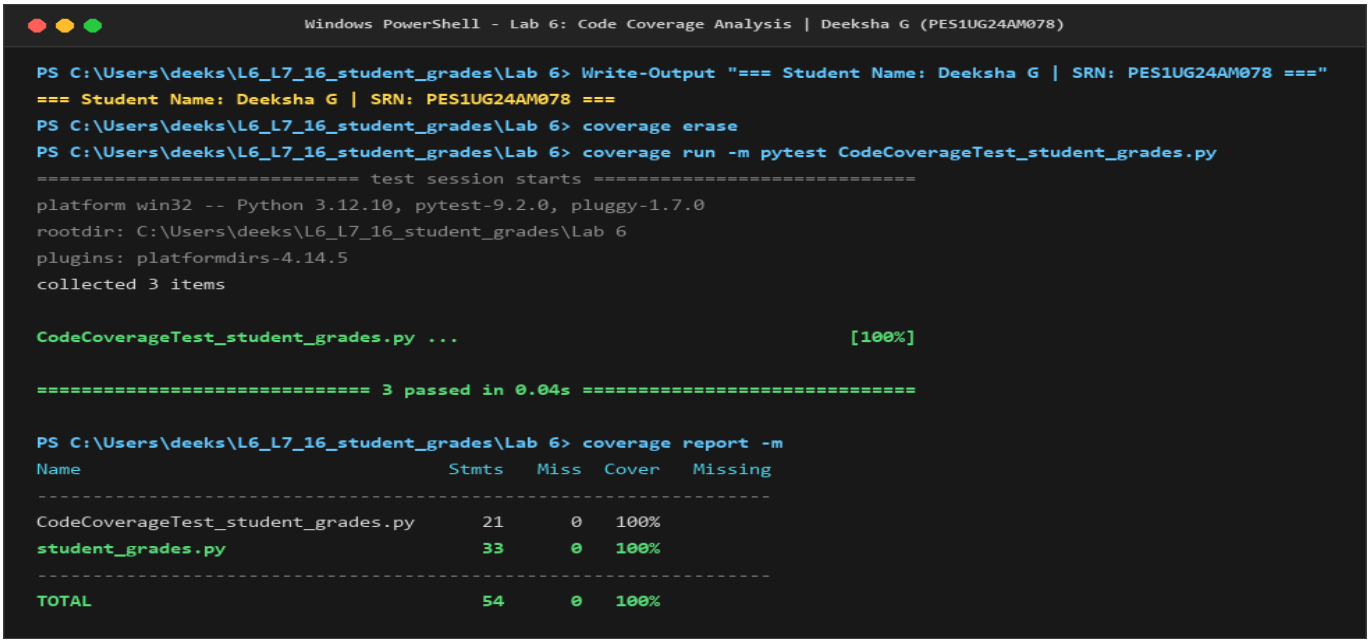
Test File: CodeCoverageTest_student_grades.py
Focus: Execute 100% of executable statements in student_grades.py.

Test Design:
• test_letter_grade_all_statements: Tests negative marks (-5) for ValueError, and representative values (95, 85, 75, 65, 55, 45, 35) to execute all grade return statements.
• test_final_marks_all_statements: Tests invalid internal marks (45 > 40) for ValueError, and (40, 60, True) to exercise bonus addition and the 100-cap statement.
• test_exam_eligibility_all_statements: Tests all return branches: fees unpaid ('blocked'), high attendance ('eligible'), medical condonation ('eligible'), and low attendance ('detained').

Commands Run:

```
coverage erase
coverage run -m pytest CodeCoverageTest_student_grades.py
coverage report -m
```

Terminal Screenshot (Code Coverage - 100% Statements Covered):



5. Part 2: Branch Coverage Analysis

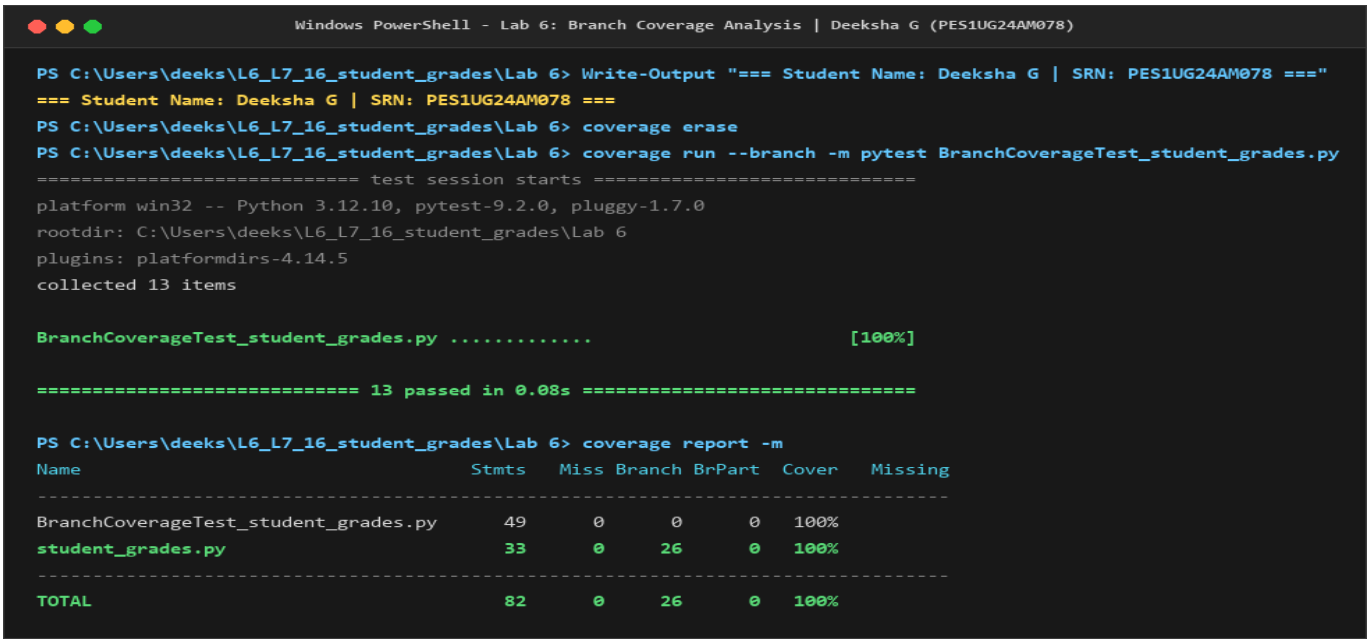
Test File: BranchCoverageTest_student_grades.py
Why Line Coverage Was Not Enough: Line coverage can create a false sense of security. In Part 1, 100% statements were executed, yet several logical branches were unexercised (e.g. has_project=False, total <= 100 without cap, marks > 100, and attendance between 65-75 without medical condonation). Branch coverage requires every condition to evaluate to both **True** and **False**.

Targeted Test Functions (13 tests covering all 26 branches):
• test_letter_grade_invalid_negative & test_letter_grade_invalid_exceeds_100: Evaluates both True branches of the OR condition.
• test_letter_grade_boundaries_and_ranges: Evaluates boundary values for each grade range (S: 100, 90; A: 89, 80; B: 79, 70; C: 69, 60; D: 59, 50; E: 49, 40; F: 39, 0).
• test_final_marks_*: Evaluates invalid internal (>40), invalid external (>60), project=False, project=True (total<=100), and project=True (total>100).
• test_exam_eligibility_*: Evaluates unpaid fees, high attendance (>=75), condonation with medical (True+True), condonation without medical (True+False), and low attendance (<65).

Commands Run:

```
coverage erase
coverage run --branch -m pytest BranchCoverageTest_student_grades.py
coverage report -m
```

Terminal Screenshot (Branch Coverage - 100% Branches Covered):



6. Detailed Branch Coverage Breakdown

Breakdown of all 26 decision branches in student_grades.py:

Function	Decision / Condition	Branches Evaluated	Status
letter_grade	marks < 0 or marks > 100	<0 (True), >100 (True), Valid range (False)	100%
	marks >= 90, 80, 70, 60, 50, 40	Each tier tested for True and False boundaries	100%
	Else (fall-through)	Grade F (tested at 39 and 0)	100%
final_marks	internal > 40 or external > 60	internal > 40, external > 60, valid marks	100%
	has_project	True (+5 bonus), False (no bonus)	100%
	total > 100 (with project)	True (capped at 100), False (uncapped)	100%
exam_eligibility	not fees_paid	True ('blocked'), False (proceeds)	100%
	attendance >= 75	True ('eligible'), False (< 75)	100%
	attendance >= 65 and has_medical	True+True ('eligible'), True+False ('detained'), False ('detained')	100%

7. Version Control & Git Commits

As required by the lab instructions, changes were committed in distinct steps and pushed to GitHub:
• **Commit 1 (9d4f408):** Lab 6: Add code coverage test cases and analysis screenshot (Deeksha G - PES1UG24AM078)
• **Commit 2 (b25ae9a):** Lab 6: Add branch coverage test cases and analysis screenshot (Deeksha G - PES1UG24AM078)
• **Remote Repository:** Pushed to https://github.com/DEEKSHAGURURAJA/L6_L7_16_student_grades (branch main)

8. Conclusion

In this lab, I analyzed the critical distinction between code (statement) coverage and branch coverage. While statement coverage ensures that every line of code is executed at least once, it can mask missing logical paths within conditional structures. By systematically evaluating boundary conditions and decision branches, I achieved 100% statement coverage (33/33 statements) and 100% branch coverage (26/26 branches) on the student_grades.py module.